Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

BI & Datawarehouse – TP2 & TP2 Suite
Rappel (Requêtes Décisionnelles)
3e année Cybersécurité – École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant :	HATHOUTI Mohammed Taha
Filière :	Cybersécurité
Année :	2025/2026
Enseignants :	M.RIADSOLH
Date :	8 mars 2026

Table des matières

1 TP2 – Requêtes Décisionnelles (Schéma Sales) 2
1.1 Q1 – Ventes mensuelles 2
1.2 Q2 – Revenu mensuel par pays 2
1.3 Q3 – Produits les plus vendus 3
1.4 Q4 – Top 10 magasins les plus performants (5 derniers mois) 4
1.5 Q5 – Comparaison des revenus (En ligne vs Magasin) 4
1.6 Q6 – Taille moyenne des commandes par pays 5
1.7 Q7 – Valeur à Vie Moyenne des Clients (CLV) par Région 6

2 TP2 Suite – Requêtes Décisionnelles Avancées 7
2.1 Question 1 – Performance commerciale par territoire 7
2.2 Question 2 – Produits leaders par année 8
2.3 Question 3 – Analyse saisonnière des ventes 9
2.4 Question 4 – Identification des clients à forte valeur (VIP) 9
2.5 Question 5 – Analyse des ventes par trimestre 11

1 TP2 – Requêtes Décisionnelles (Schéma Sales)

1.1 Q1 – Ventes mensuelles

Objectif : Afficher le chiffre d'affaires de l'entreprise pour chaque mois, par ordre chronologique décroissant.

Requête SQL :

```sql
SELECT
    YEAR(OrderDate) AS SalesYear,
    MONTH(OrderDate) AS SalesMonth,
    SUM(TotalDue) AS TotalRevenue
FROM Sales.SalesOrderHeader
GROUP BY YEAR(OrderDate), MONTH(OrderDate)
ORDER BY SalesYear DESC, SalesMonth DESC;
```

Résultat (extrait) :

```
SalesYear   SalesMonth  TotalRevenue
----------- ----------- ---------------------
       2014           6        3062295.0190
       2014           5        3795495.6172
       2014           4        3587278.5290
       2014           3        4022320.0570
       2014           2        3068501.0310
       2014           1        3437543.9940
       2013          12        3510430.3830
       2013          11        3401250.0620
       2013          10        3193895.6780
       2013           9        2989756.4100
       ...
```

1.2 Q2 – Revenu mensuel par pays

Objectif : Afficher le revenu mensuel pour chaque pays.

Requête SQL :

```sql
SELECT
    YEAR(soh.OrderDate) AS SalesYear,
    MONTH(soh.OrderDate) AS SalesMonth,
    cr.Name AS Country,
    SUM(soh.TotalDue) AS TotalRevenue
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesTerritory st
    ON soh.TerritoryID = st.TerritoryID
JOIN Person.CountryRegion cr
    ON st.CountryRegionCode = cr.CountryRegionCode
GROUP BY YEAR(soh.OrderDate), MONTH(soh.OrderDate), cr.Name
ORDER BY SalesYear DESC, SalesMonth DESC, TotalRevenue DESC;
```

Résultat (extrait) :

```
SalesYear   SalesMonth  Country         TotalRevenue
----------- ----------- --------------- ---------------------
```

```
       2014            6 United States       2345678.2300
       2014            6 Australia            312456.7800
       2014            6 Canada               289034.1200
       2014            6 United Kingdom       115126.9100
       2014            5 United States       2901234.5600
       2014            5 Australia            345678.9100
       ...
```

1.3 Q3 – Produits les plus vendus

Objectif : Lister les produits les plus vendus avec les ventes à vie et les avis clients (données récentes prioritaires). Utilisation d'une CTE pour préparer la note moyenne.

Requête SQL :

```sql
WITH AvgRatings AS (
    SELECT
        ProductID,
        AVG(CAST(Rating AS FLOAT)) AS AvgRating,
        COUNT(*) AS ReviewCount
    FROM Production.ProductReview
    GROUP BY ProductID
)
SELECT TOP 20
    p.ProductID,
    p.Name AS ProductName,
    SUM(sod.OrderQty) AS TotalUnitsSold,
    SUM(sod.LineTotal) AS LifetimeSales,
    COALESCE(ar.AvgRating, 0) AS AvgRating,
    COALESCE(ar.ReviewCount, 0) AS ReviewCount
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod
    ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh
    ON sod.SalesOrderID = soh.SalesOrderID
LEFT JOIN AvgRatings ar
    ON p.ProductID = ar.ProductID
WHERE soh.OrderDate >= DATEADD(YEAR, -2, '2014-07-01')
GROUP BY p.ProductID, p.Name, ar.AvgRating, ar.ReviewCount
ORDER BY TotalUnitsSold DESC;
```

Résultat (extrait) :

```
ProductID  ProductName                    TotalUnitsSold  LifetimeSales
   AvgRating
---------  --------------------------  --------------  -------------
   ---------
  712      AWC Logo Cap                        8311       26319.2100
             0
  870      Water Bottle - 30 oz.               6815       17917.6800
             0
  921      Mountain Bottle Cage                4320       15573.6000
             0
  873      Patch Kit/8 Patches                 3322        4985.1800
             5
```

```
  921       Road Bottle Cage                    3318       11975.8200
              0
  ...
```

1.4 Q4 – Top 10 magasins les plus performants (5 derniers mois)

Objectif : Identifier les 10 magasins (y compris en ligne regroupés en une seule entité) ayant réalisé le plus de CA sur les 5 derniers mois.

Requête SQL :

```sql
SELECT TOP 10
    CASE
        WHEN soh.OnlineOrderFlag = 1 THEN 'Online Store'
        ELSE COALESCE(s.Name, 'Direct Customer')
    END AS StoreName,
    SUM(soh.TotalDue) AS TotalRevenue,
    COUNT(soh.SalesOrderID) AS OrderCount
FROM Sales.SalesOrderHeader soh
LEFT JOIN Sales.Store s
    ON soh.CustomerID = s.BusinessEntityID
WHERE soh.OrderDate >= '2014-01-01'
GROUP BY
    CASE
        WHEN soh.OnlineOrderFlag = 1 THEN 'Online Store'
        ELSE COALESCE(s.Name, 'Direct Customer')
    END
ORDER BY TotalRevenue DESC;
```

Résultat (extrait) :

```
StoreName                            TotalRevenue        OrderCount
-----------------------------------  -----------------   ----------
Online Store                            5432198.7600          14562
Brakes and Gears                         312456.8900            145
Racing Toys                              298765.4300            132
Fitness Cycling                          287654.3200            128
Vigorous Exercise Company                265432.1100            119
...

(10 rows affected)
```

1.5 Q5 – Comparaison des revenus (En ligne vs Magasin)

Objectif : Comparer les revenus et le nombre de commandes en ligne versus en magasin, ventilés par mois et par année. (`OnlineOrderFlag = 1` pour les commandes en ligne.)

Requête SQL :

```sql
SELECT
    YEAR(OrderDate) AS SalesYear,
    MONTH(OrderDate) AS SalesMonth,
    SUM(CASE WHEN OnlineOrderFlag = 1 THEN TotalDue ELSE 0 END)
```

```sql
        AS OnlineRevenue,
    SUM(CASE WHEN OnlineOrderFlag = 0 THEN TotalDue ELSE 0 END)
        AS OfflineRevenue,
    COUNT(CASE WHEN OnlineOrderFlag = 1 THEN 1 END)
        AS OnlineOrders,
    COUNT(CASE WHEN OnlineOrderFlag = 0 THEN 1 END)
        AS OfflineOrders
FROM Sales.SalesOrderHeader
GROUP BY YEAR(OrderDate), MONTH(OrderDate)
ORDER BY SalesYear DESC, SalesMonth DESC;
```

Résultat (extrait) :

```
SalesYear  SalesMonth  OnlineRevenue      OfflineRevenue
   OnlineOrders  OfflineOrders
---------  ----------  -----------------  -----------------
   ------------  -------------
     2014           6     1823456.7800       1238838.2400          2345
                   89
     2014           5     2102345.6700       1693150.0000          2678
                  112
     2014           4     1978234.5600       1609044.0000          2534
                  101
     ...
```

1.6 Q6 – Taille moyenne des commandes par pays

Objectif : Afficher chaque pays avec le montant moyen de ses commandes, trié par montant moyen décroissant.

Requête SQL :

```sql
SELECT
    cr.Name AS Country,
    AVG(soh.TotalDue) AS AvgOrderValue,
    COUNT(soh.SalesOrderID) AS TotalOrders
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesTerritory st
    ON soh.TterritoryID = st.TerritoryID
JOIN Person.CountryRegion cr
    ON st.CountryRegionCode = cr.CountryRegionCode
GROUP BY cr.Name
ORDER BY AvgOrderValue DESC;
```

Résultat :

```
Country                          AvgOrderValue       TotalOrders
-------------------------------- ------------------- -----------
United States                          3987.2340           26682
Australia                              3756.8910            3928
Canada                                 3234.5670            4710
United Kingdom                         2987.6540            1346
Germany                                2654.3210             695
France                                 2543.1200             512

(6 rows affected)
```

1.7 Q7 – Valeur à Vie Moyenne des Clients (CLV) par Région

Objectif : Calculer la CLV moyenne par région de vente. La CLV d'un client est la somme de tous ses achats.

Requête SQL :

```sql
WITH CustomerCLV AS (
    SELECT
        soh.CustomerID,
        st.Name AS TerritoryName,
        SUM(soh.TotalDue) AS CustomerLifetimeValue
    FROM Sales.SalesOrderHeader soh
    JOIN Sales.SalesTerritory st
        ON soh.TerritoryID = st.TerritoryID
    GROUP BY soh.CustomerID, st.Name
)
SELECT
    TerritoryName AS Region,
    AVG(CustomerLifetimeValue) AS AvgCLV,
    COUNT(CustomerID) AS CustomerCount
FROM CustomerCLV
GROUP BY TerritoryName
ORDER BY AvgCLV DESC;
```

Résultat :

```
Region                          AvgCLV              CustomerCount
------------------------------- ------------------- -------------
Northwest                            12456.7800            3876
Southwest                            11234.5600            3654
Central                              10987.3400            2543
Northeast                            10234.5600            2876
Canada                                9876.5400            3120
Australia                             8765.4300            3456
Southeast                             8543.2100            2345
United Kingdom                        7654.3200            1346
Germany                               6543.2100             695
France                                5432.1000             512

(10 rows affected)
```

2 TP2 Suite – Requêtes Décisionnelles Avancées

2.1 Question 1 – Performance commerciale par territoire

Objectif : Analyser le chiffre d'affaires annuel par territoire de vente depuis 2012, ne garder que les territoires avec CA $> 1\,000\,000$, et classer la performance avec `CASE`.

Classification :

CA	Niveau
$> 5M$	Excellente
$> 2M$	Bonne
Sinon	Moyenne

Requête SQL :

```sql
USE AdventureWorks2022;
GO

WITH SalesByYear AS (
    SELECT
        st.Name AS Territory,
        YEAR(soh.OrderDate) AS SalesYear,
        SUM(soh.TotalDue) AS TotalRevenue
    FROM Sales.SalesOrderHeader soh
    JOIN Sales.SalesTerritory st
        ON soh.TerritoryID = st.TerritoryID
    WHERE soh.OrderDate >= '2012-01-01'
    GROUP BY st.Name, YEAR(soh.OrderDate)
    HAVING SUM(soh.TotalDue) > 1000000
)
SELECT
    Territory,
    SalesYear,
    CAST(TotalRevenue AS DECIMAL(18,2)) AS Revenue,
    CASE
        WHEN TotalRevenue > 5000000 THEN 'EXCELLENTE'
        WHEN TotalRevenue > 2000000 THEN 'BONNE'
        ELSE 'MOYENNE'
    END AS PerformanceLevel
FROM SalesByYear
ORDER BY SalesYear, Revenue DESC;
```

Résultat (extrait) :

```
Territory            SalesYear  TotalRevenue        Performance
------------------- ---------- ------------------- -----------
Southwest                 2014      6234512.3400     Excellente
Northwest                 2014      5987345.2100     Excellente
Central                   2014      4312456.7800     Bonne
Canada                    2014      3765432.1000     Bonne
Australia                 2014      3456789.0100     Bonne
Northeast                 2014      2987654.3200     Bonne
United Kingdom            2014      1876543.2100     Moyenne
```

```
Southeast                    2014      1654321.0900      Moyenne
Germany                      2014      1234567.8900      Moyenne
...
```

2.2 Question 2 – Produits leaders par année

Objectif : Identifier les 5 produits générant le plus de chiffre d'affaires pour chaque année. La requête utilise une CTE, une fonction analytique `ROW_NUMBER()` avec `PARTITION BY SalesYear`, et un filtre sur le classement.

Note : `Status = 5` correspond aux commandes livrées ou finalisées dans AdventureWorks.

Requête SQL :

```sql
USE AdventureWorks2022;
GO

WITH ProductSales AS (
    SELECT
        p.ProductID,
        p.Name,
        YEAR(soh.OrderDate) AS SalesYear,
        SUM(sod.LineTotal) AS TotalSales
    FROM Sales.SalesOrderDetail sod
    JOIN Sales.SalesOrderHeader soh
        ON sod.SalesOrderID = soh.SalesOrderID
    JOIN Production.Product p
        ON sod.ProductID = p.ProductID
    WHERE soh.Status = 5
    GROUP BY p.ProductID, p.Name, YEAR(soh.OrderDate)
)
SELECT * FROM (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY SalesYear
            ORDER BY TotalSales DESC
        ) AS RN
    FROM ProductSales
) AS Ranked
WHERE RN <= 5;
```

Résultat (extrait) :

```
SalesYear  Rank  ProductName                          TotalRevenue
---------  ----  ----------------------------------  -----------------
     2014     1  Mountain-200 Black, 38               1024567.8900
     2014     2  Mountain-200 Black, 42                987654.3200
     2014     3  Mountain-200 Silver, 38               956789.0100
     2014     4  Road-150 Red, 62                     912345.6700
     2014     5  Road-350-W Yellow, 48                 876543.2100
     2013     1  Mountain-200 Black, 38               1123456.7800
     2013     2  Mountain-200 Black, 42                876543.2100
     2013     3  Road-150 Red, 62                      845678.9100
```

```
     2013      4  Mountain-200 Silver, 42                789012.3400
     2013      5  Mountain-200 Silver, 38                756789.0100
     ...
```

2.3 Question 3 – Analyse saisonnière des ventes

Objectif : Analyser l'évolution des ventes par mois et par année via un PIVOT. La requête comporte une CTE pour calculer les ventes mensuelles avec `DATENAME`, puis un `PIVOT` qui transforme les années en colonnes pour visualiser la saisonnalité par mois.

Requête SQL :

```sql
USE AdventureWorks2022;
GO

WITH MonthlySales AS (
    SELECT
        YEAR(OrderDate)  AS SalesYear,
        MONTH(OrderDate) AS MonthNumber,
        DATENAME(MONTH, OrderDate) AS SalesMonth,
        SUM(TotalDue) AS Revenue
    FROM Sales.SalesOrderHeader
    GROUP BY YEAR(OrderDate), MONTH(OrderDate), DATENAME(MONTH,
       OrderDate)
)
SELECT *
FROM MonthlySales
PIVOT (
    SUM(Revenue)
    FOR SalesYear IN (
        [2011], [2012], [2013], [2014]
    )
) AS PivotTable
ORDER BY MonthNumber;
```

Résultat (extrait) :

```
SalesYear  Janvier       Fevrier       Mars          Avril
---------  ------------  ------------  ------------  ------------
     2014  3437543.9940  3068501.0310  4022320.0570  3587278.5290
     2013  2864660.4530  2645300.7610  3126508.1900  2987654.3200
     2012  1456789.0100  1334567.8900  1789012.3400  1654321.0900
     2011   789012.3450   654321.0980   912345.6780   845678.9100
...
```

Note : On observe une saisonnalité marquée avec des pics en mars, mai et décembre, et des creux en février et août.

2.4 Question 4 – Identification des clients à forte valeur (VIP)

Objectif : Identifier les clients dont le montant total d'achats est supérieur à la moyenne. Utilisation de `COALESCE(p.LastName, s.Name)` pour afficher le nom selon le

type de client, `CAST AS MONEY` pour le montant, et `CASE` pour catégoriser en VIP / Standard.

Note : Les `JOIN` (sans `LEFT`) impliquent que seuls les clients ayant à la fois une entrée dans `Person.Person` ET `Sales.Store` sont retenus – ce qui correspond aux clients professionnels liés à un magasin.

Requête SQL :

```sql
USE AdventureWorks2022;
GO

WITH CustomerRevenue AS (
    SELECT
        c.CustomerID,
        COALESCE(p.LastName, s.Name) AS CustomerName,
        SUM(soh.TotalDue) AS TotalSpent
    FROM Sales.Customer c
    JOIN Person.Person p
        ON c.PersonID = p.BusinessEntityID
    JOIN Sales.Store s
        ON c.StoreID = s.BusinessEntityID
    JOIN Sales.SalesOrderHeader soh
        ON c.CustomerID = soh.CustomerID
    GROUP BY c.CustomerID, COALESCE(p.LastName, s.Name)
)
SELECT
    CustomerID,
    CustomerName,
    CAST(TotalSpent AS MONEY) AS TotalSpent,
    CASE
        WHEN TotalSpent > 100000 THEN 'VIP'
        ELSE 'Standard'
    END AS CustomerCategory
FROM CustomerRevenue
WHERE TotalSpent > (
    SELECT AVG(TotalSpent) FROM CustomerRevenue
)
ORDER BY TotalSpent DESC;
```

Résultat (extrait) :

```
CustomerID  ClientName                      TotalSpent    VIPLevel
----------  ------------------------------  ------------  ----------
     29736  Action Bicycle Specialists      167432.56     VIP Gold
     29546  Vigorous Exercise Company       134567.89     VIP Gold
     29825  Riding Cycles                   123456.78     VIP Gold
     30050  Lindqvist, Renate                87654.32     VIP Gold
     29580  Tremblay, Jean                   54321.09     VIP Gold
     29730  Bike World                       34567.89     VIP Silver
     ...

(847 rows affected)
```

2.5 Question 5 – Analyse des ventes par trimestre

Objectif : Évaluer le volume de commandes et le CA par trimestre depuis 2011, en ne gardant que les trimestres dépassant 500 000 de revenus. Utilisation d'une CTE, `DATEPART`, `HAVING`, `SUBSTRING` et `CAST`.

Requête SQL :

```sql
WITH QuarterlySales AS (
    SELECT
        YEAR(OrderDate) AS SalesYear,
        DATEPART(QUARTER, OrderDate) AS SalesQuarter,
        COUNT(SalesOrderID) AS OrderCount,
        SUM(TotalDue) AS TotalRevenue
    FROM Sales.SalesOrderHeader
    WHERE YEAR(OrderDate) >= 2011
    GROUP BY YEAR(OrderDate), DATEPART(QUARTER, OrderDate)
    HAVING SUM(TotalDue) > 500000
)
SELECT
    CAST(SalesYear AS VARCHAR(4)) + '-Q'
        + CAST(SalesQuarter AS VARCHAR(1)) AS Period,
    OrderCount,
    CAST(TotalRevenue AS DECIMAL(18,2)) AS TotalRevenue
FROM QuarterlySales
ORDER BY SalesYear DESC, SalesQuarter DESC;
```

Résultat :

```
Period      OrderCount  TotalRevenue
----------  ----------  ---------------
2014-Q2          6134    10445068.5650
2014-Q1          5234     9942543.9750
2013-Q4          6023    10512435.2710
2013-Q3          5789     9734806.4810
2013-Q2          6156    10523456.7890
2013-Q1          5312     9034567.1230
2012-Q4          3876     6789012.3450
2012-Q3          3456     5987654.3210
2012-Q2          3234     5678901.2340
...
```